# Robotic Arm Gripping + Multimodal Visual Understanding + SLAM Navigation

## 1. Course Content

- After running the example program, combine Nav2 navigation, robotic arm gripping, and AI large-model visual understanding to perform complex task functions
- **Note: this chapter requires the map mapping file to be configured first**
- **This case uses the matching 3 × 3 × 3 blocks**

> **Note**
>
> - The difference between the text version and the voice version lies in the command input method, and the text version does not require the speech recognition and speech synthesis stages

## 2. Preparation

### 2.1 Content Description

> This lesson applies to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You must first enter the Docker container before running ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in the [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Run commands directly in the onboard terminal; there is no need to enter Docker.

### 2.2 Start the Local Dify Large-Model Environment

Enter the following command in the host terminal to start the Dify service:

```
sh ~/bringup_dify.sh
```

The terminal prints the following message, indicating that the Dify service is connected successfully:



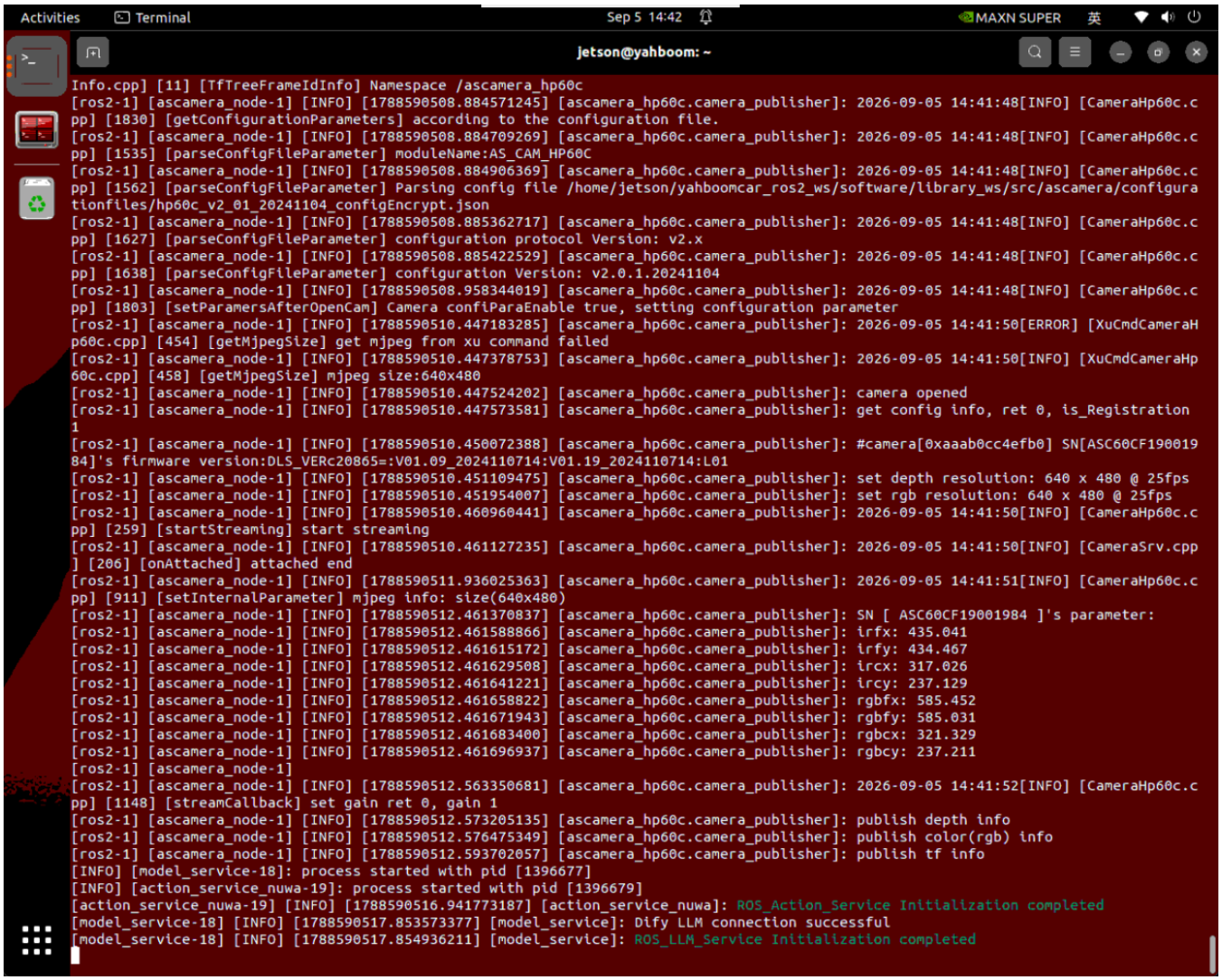

> **Important**
>
> **Precondition**:
> This lesson requires the robot to navigate on an already-built map. Please make sure that a grid map (such as `map.yaml`) has been built and saved according to the navigation chapter tutorial.

# 3. Running Examples

## 3.1 Start the System Master Control Node (Text Interaction Mode)

Enter the following command in the onboard terminal to start the large-model embodied control service:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```




## 3.2 Start the SLAM Navigation and Visualization System

On the onboard desktop or a virtual machine, open another terminal and start the navigation visualization interface:

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

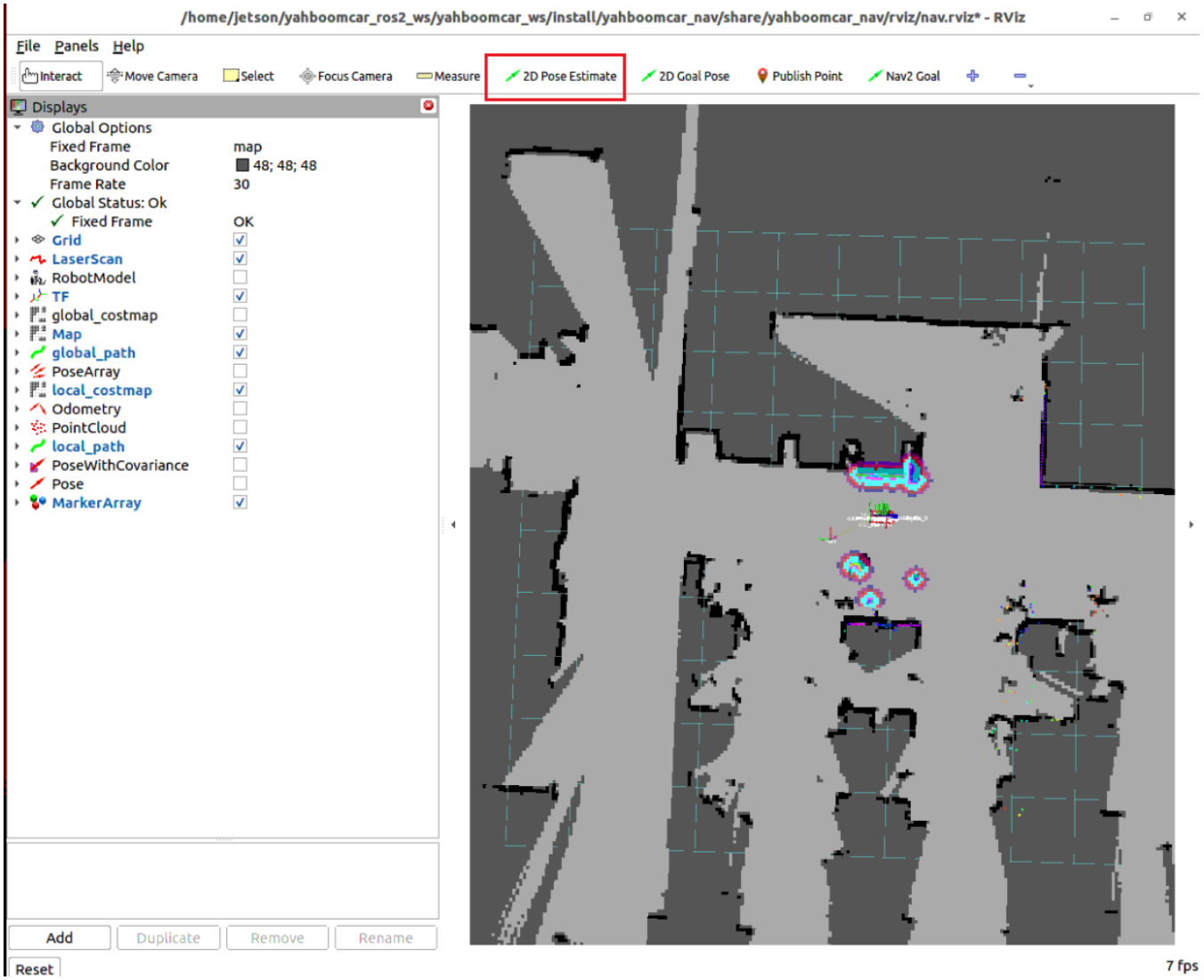
Then open another terminal and start the underlying navigation algorithm **(the navigation algorithm must match the localization method used during waypoint calibration; if relocalization was used, run the relocalization navigation, because the poses of different localization methods may differ)**:

```
# AMCL localization + navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

# Relocalization method; see the Cartographer relocalization navigation for details
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pbst
ream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
```

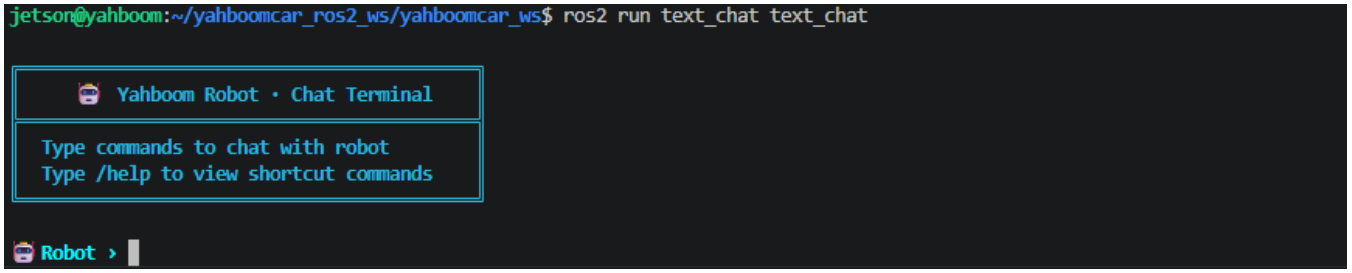
In the opened RViz2 window, use the **2D Pose Estimate** tool to perform initial alignment and localization according to the robot's parked position in the real scene.




## 3.3 Start the Text Interaction Terminal

Enter the following command in the SSH terminal on the computer to start the text interaction interface:

```
ros2 run text_chat text_chat
```




## 3.4 Test Cases

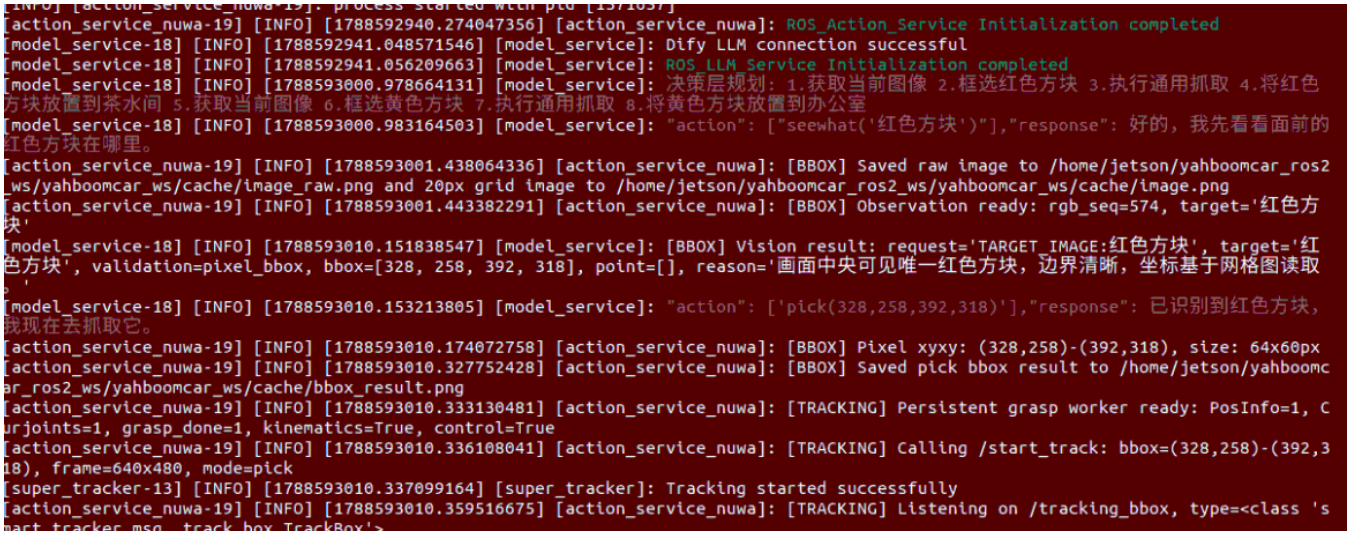
Two reference test cases are given here; users can write their own test commands

- Blocks used: 30x30x30 mm blocks.
- Help me take the red block in front of you to the tea room, and then take the yellow block in the tea room to the office

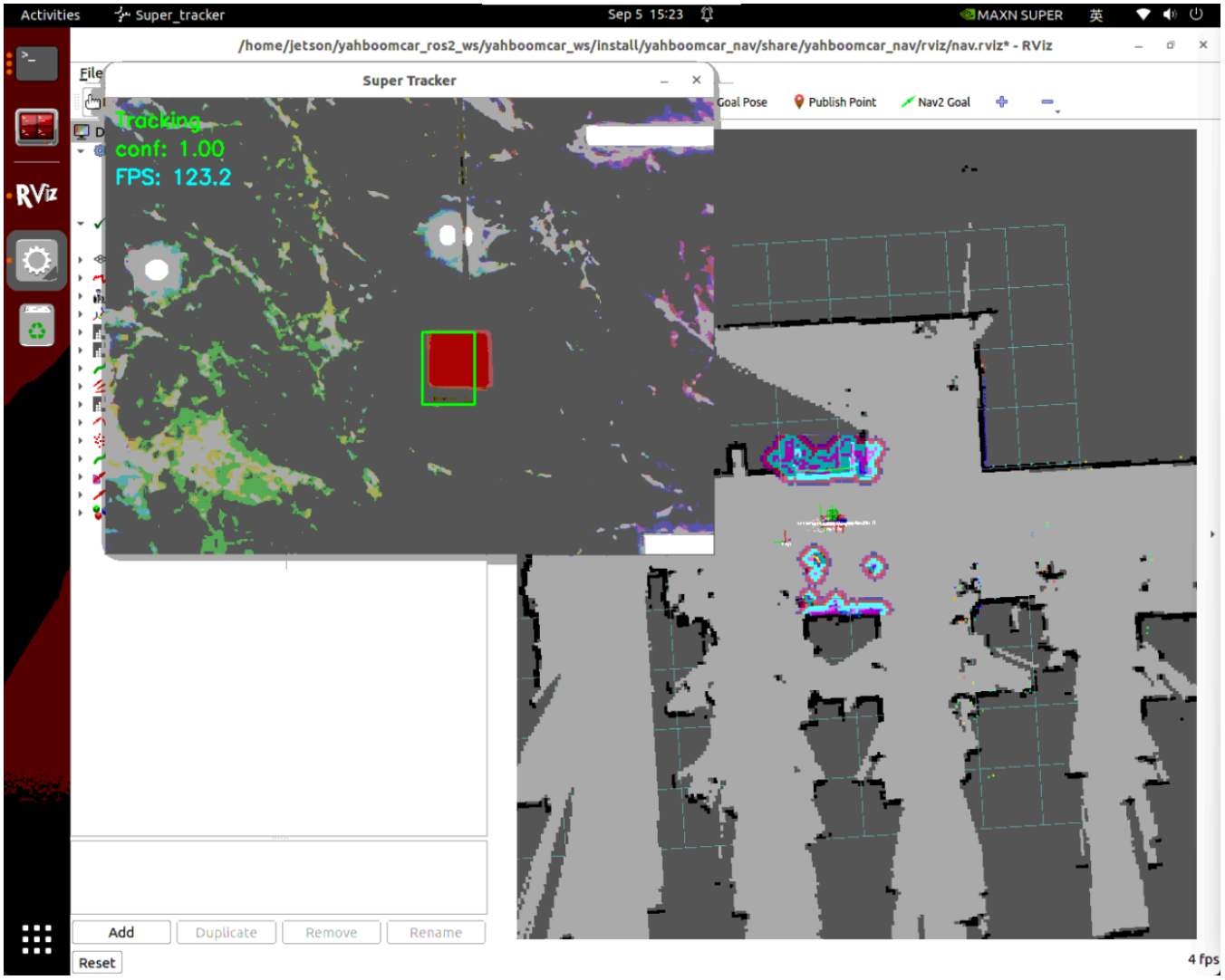
Enter the test case in the text interaction terminal, and the decision-layer large model outputs the planned task steps:




The execution-layer large model then executes according to these task steps:



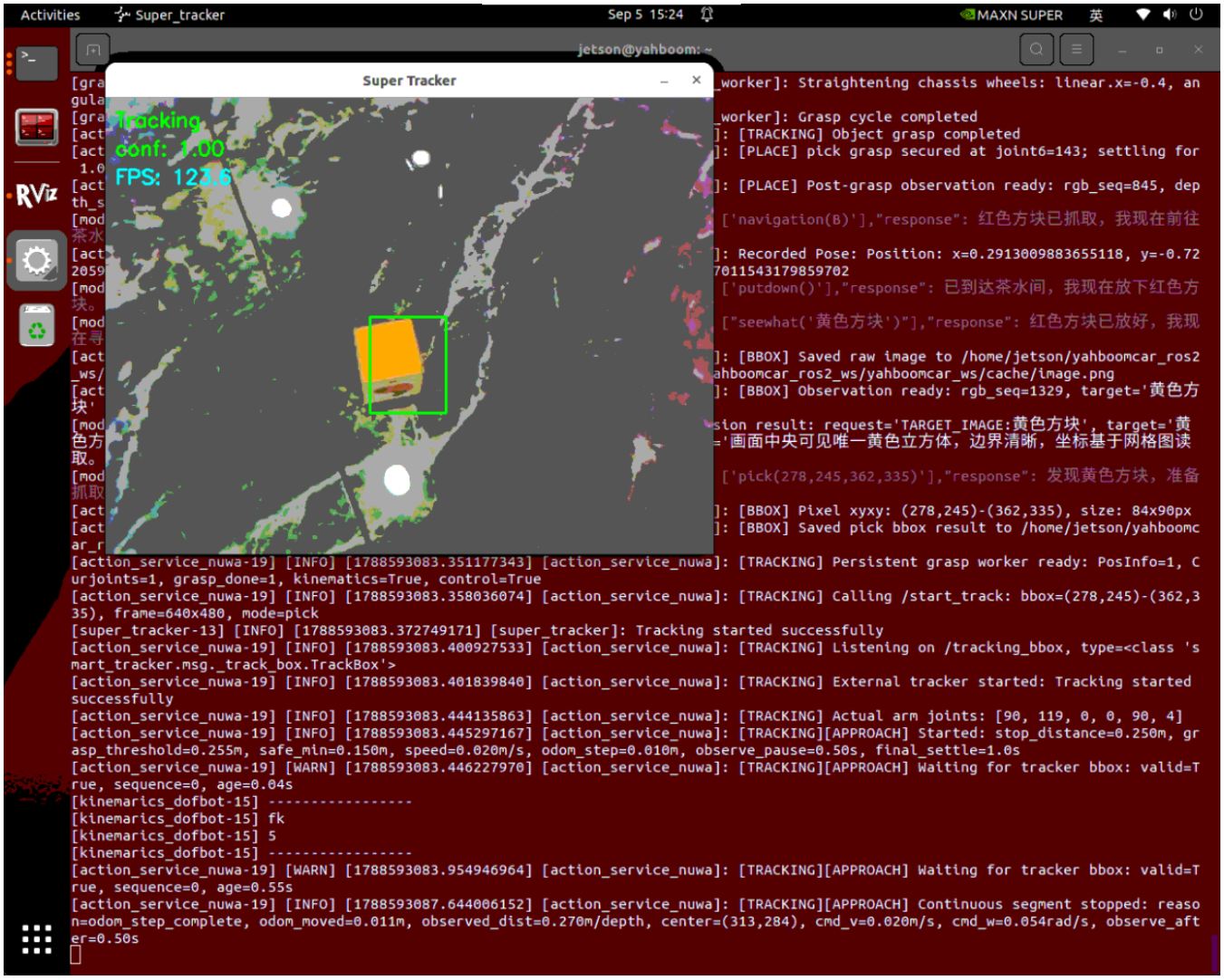

The robot first grasps the red block in front of it as required by the command




Then it navigates to the "tea room" and uses the robotic arm to put down the red block

```
[INFO] [1788591460.416590258] [text_chat_node]: Received response: 我看到红色方块了，现在去抓取它。
  Robot: 红色方块已抓取，现在前往茶水间。
  Action: navigation(B)
[INFO] [1788591491.976858427] [text_chat_node]: Received response: 红色方块已抓取，现在前往茶水间。
  Robot: 已到达茶水间，现在放下红色方块。
  Action: putdown()
[INFO] [1788591510.749152415] [text_chat_node]: Received response: 已到达茶水间，现在放下红色方块。
  Robot: 红色方块已放好，现在寻找茶水间的黄色方块。
  Action: seewhat()
```

After re-observing and finding and grasping the green block in the "master bedroom," it heads to the "office"




After putting down the green block in the "office," the robot indicates that the task is complete and enters the waiting state; at this point, enter "you can rest now, let the robot end the task" in the interaction terminal

```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 run text_chat text_chat
获取当前图像 6.框选黄色方块 7.执行通用抓取 8.将黄色方块放置到办公室
  Robot: 好的，我先看看面前的红色方块在哪里。
  Action: ["seewhat('红色方块')"]
[INFO] [1788593001.092887770] [text_chat_node]: Received response: 好的，我先看看面前的红色方块在哪里。
  Robot: 已识别到红色方块，我现在去抓取它。
  Action: pick(328,258,392,318)
[INFO] [1788593010.155594330] [text_chat_node]: Received response: 已识别到红色方块，我现在去抓取它。
  Robot: 红色方块已抓取，我现在前往茶水间。
  Action: navigation(B)
[INFO] [1788593035.558804833] [text_chat_node]: Received response: 红色方块已抓取，我现在前往茶水间。
  Robot: 已到达茶水间，我现在放下红色方块。
  Action: putdown()
[INFO] [1788593061.867571098] [text_chat_node]: Received response: 已到达茶水间，我现在放下红色方块。
  Robot: 红色方块已放好，我现在寻找茶水间的黄色方块。
  Action: ["seewhat('黄色方块')"]
[INFO] [1788593073.432622062] [text_chat_node]: Received response: 红色方块已放好，我现在寻找茶水间的黄色方块。
  Robot: 发现黄色方块，准备抓取。
  Action: pick(278,245,362,335)
[INFO] [1788593083.254742912] [text_chat_node]: Received response: 发现黄色方块，准备抓取。
  Robot: 黄色方块已抓取，我现在前往办公室。
  Action: navigation(A)
[INFO] [1788593109.245479300] [text_chat_node]: Received response: 黄色方块已抓取，我现在前往办公室。
  Robot: 已到达办公室，我现在放下黄色方块。
  Action: putdown()
[INFO] [1788593147.503748685] [text_chat_node]: Received response: 已到达办公室，我现在放下黄色方块。
  Robot: 任务完成，红色方块已送至茶水间，黄色方块已送至办公室。
[INFO] [1788593159.604610714] [text_chat_node]: Received response: 任务完成，红色方块已送至茶水间，黄色方块已送至办公室。
```